

Python RAG Engineering Guide

Implementation-oriented reference for building a document Q&A; pipeline

1. Recommended Project Structure

```
rag-doc-copilot/  
  data/pdfs/  
  src/  
    ingestion.py  
    embeddings.py  
    retrieval.py  
    app.py  
  chroma_db/  
  .env  
  requirements.txt
```

Separating ingestion, indexing, retrieval, and interface code makes the application easier to test and maintain.

2. PDF Loading

PyPDFLoader can load PDF pages into LangChain Document objects. Each Document contains `page_content` and `metadata`. The `source` and `page` fields are especially useful when displaying evidence to users.

```
loader = PyPDFLoader("data/pdfs/manual.pdf")
documents = loader.load()
```

3. Recursive Chunking

`RecursiveCharacterTextSplitter` attempts to preserve natural text boundaries by recursively applying separators. For the initial project configuration, use `chunk_size` 500 and `chunk_overlap` 50.

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50,  
)  
chunks = splitter.split_documents(documents)
```

4. Retrieval Function

The retrieval function should accept a user question and return a small set of relevant Documents. A top-k value of 3 keeps the generation context focused.

```
def retrieve(query, vector_store):  
    return vector_store.similarity_search(query, k=3)
```

The function should not generate an answer. Retrieval and generation should remain separate so each layer can be evaluated independently.

5. Prompt Construction

The prompt should contain the retrieved passages and the user question. Clear instructions reduce the chance that the model relies on unsupported prior knowledge.

```
prompt = f"""
Use only the context below to answer.
If the answer is not present, say so.
```

```
Context:
{context}
```

```
Question:
{question}
"""
```

6. Configuration and Security

API keys should never be hard-coded into source files or committed to Git. Store secrets in environment variables, commonly loaded from a local .env file during development.

```
from dotenv import load_dotenv
load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")
```